Building apps for Tactility

What an app actually is

An app is **not** a C++ class. It's a C-style API (`<app/*.h>`) with two pieces:

1. An `AppManifest` — metadata describing the app (`id`, `name`, `category`, `location`, `flags`).
2. An `AppMainFn` **entry point** — `int32_t appMain(uint32_t appInstanceId, int argc, char* argv[])`, modelled on a C program's `main()`.

Every app instance gets its **own dedicated FreeRTOS task for its entire lifetime**, and that task blocks in `appMain` until the app returns.

There are two deployment models:

Model	Location type	Where it runs from
Built-in app	`APP_LOCATION_MEMORY`	Compiled into the firmware; `location.location` holds the function pointer
External app	`APP_LOCATION_PATH`	An installed folder or `.elf` binary on SD/flash, loaded by an `AppLoaderApi`

Most people start with a **built-in app**, so that's what this guide focuses on.

The anatomy of a built-in app

Built-in apps live at `Tactility/Source/app/<appname>/<AppName>.cpp`, one folder per app. The cleanest minimal reference is `Tactility/Source/app/systeminfo/SystemInfo.cpp`; the modal-dialog example `Tactility/Source/app/inputdialog/InputDialog.cpp` shows parameters and results.

1. Manifest

```
extern const ::AppManifest manifest = {
    .id = "SystemInfo",
    .name = "System Info",
    .category = APP_CATEGORY_SYSTEM,
    .location = { APP_LOCATION_MEMORY, reinterpret_cast<void*>(appMain) }
};
```

Fields (see `Modules/app-module/include/app/manifest.h`):

- **`id`** — unique string; used to launch/route to the app. Must never be `NULL`.
- **`name`** — human-readable, shown in UI.
- **`category`** — `APP_CATEGORY_SYSTEM`, `APP_CATEGORY_SETTINGS`, or `APP_CATEGORY_USER` (used for grouping in the launcher).
- **`location`** — `APP_LOCATION_MEMORY` + the `appMain` pointer for a built-in app.

- `flags` — bitmask of `AppManifestFlags`. `APP_MANIFEST_FLAG_HIDDEN` hides the app from generic app-browsing UIs (use it for modal dialogs, wizards, detail views).

2. Entry point and event loop

The standard shape every app follows:

```
int32_t appMain(uint32_t appInstanceId, int argc, char* argv[]) {
    Context ctx{};
    ctx.appInstanceId = appInstanceId;

    // 1. Subscribe to lifecycle events for THIS instance
    AppEventSubscription sub{};
    sub.app_instance_id = appInstanceId;
    app_event_subscribe(&sub);

    // 2. Create your window (builds widgets via your createWidgets callback)
    WindowId window = window_manager_create(appInstanceId, createWidgets, &ctx);

    // 3. Block until told to close
    bool shouldClose = false;
    while (!shouldClose) {
        AppEvent event{};
        if (app_event_await(&sub, &event, portMAX_DELAY) != ERROR_NONE) {
            break;
        }
        switch (event.type) {
            case APP_EVENT_CLOSE:
                app_manager_finish(appInstanceId);
                shouldClose = true;
                break;
            default:
                break;
        }
    }

    // 4. Tear down
    window_manager_remove(window);
    app_event_unsubscribe(&sub);
    return 0;   // 0 = Ok, 1 = Cancelled, 2 = Error (by convention)
}
```

Key APIs (all in `Modules/app-module/include/app/`):

- `app_event_subscribe()` / `app_event_await()` / `app_event_unsubscribe()` — the event loop. Events are `APP_EVENT_CLOSE` (terminate now) and `APP_EVENT_RESULT` (a modal child reported back). See `app/event.h`.
- `app_manager_finish()` — call this right before returning from your **own** `appMain` when closing yourself.

- `app_manager_stop()` — use this to close **another** instance (never call it from your own task — it would join yourself and assert).

3. UI (LVGL window manager)

UI is optional. If your app has a UI, it goes through the LVGL window-manager module (`<lvgl_window_manager/window_manager.h>`), which gives each app instance one stacked window.

- `window_manager_create(appInstanceId, createWidgets, userData)` returns a `WindowId`; `createWidgets` is your callback:

```
void createWidgets(lv_obj_t* parent, void* userData) {
    auto* ctx = static_cast<Context*>(userData);
    auto* toolbar = lvgl_toolbar_create(parent, "My App");
    lvgl_toolbar_set_nav_action(toolbar, LV_SYMBOL_CLOSE, onBackPressed,
ctx);
    // ...build your widgets as children of `parent`
}
```

- **Locking:** any task touching LVGL objects must hold the LVGL lock first — `lvgl_lock()` / `lvgl_unlock()` (from `lvgl-module`). Your `createWidgets` callback already runs on the LVGL task with the lock held, but timers and worker threads must wrap their LVGL calls.
- **Toolbar** helper: `lvgl_toolbar_create` / `lvgl_toolbar_set_nav_action` from `lvgl/widgets/toolbar.h`. Shared fonts live in `lvgl/fonts.h`.
- **Close button pattern** (important — do this, not a direct `app_manager_stop()`): in the toolbar/back callback, emit an event instead of stopping yourself:

```
void onBackPressed(lv_event_t* event) {
    auto* ctx = static_cast<Context*>(lv_event_get_user_data(event));
    AppEvent closeEvent{ .type = APP_EVENT_CLOSE, .timestamp = 0, .result =
{} };
    app_event_emit(ctx->appInstanceId, &closeEvent);
}
```

  Calling `app_manager_stop()` from an LVGL callback deadlocks (it joins your thread, which needs the LVGL lock that callback already holds).

Two critical `createWidgets` caveats (from `window_manager.h` and `lvgl.md`):

- Only the **topmost** window ever has live widgets. When a window is buried and later resurfaces, its widgets are **deleted and rebuilt** via `createWidgets` again — so `createWidgets` must only rebuild already-committed state, never decide what happens next. State transitions belong in your `appMain` event loop.
- The rebuild can run on a **different thread** than the one that created the window, so don't rely on thread-local state — use the `userData`/`Context*`.

4. Public API header (optional)

If other code needs to launch your app or read a result, add a header at `Tactility/Include/Tactility/app/<appname>/<AppName>.h`. See `InputDialog.h`:

```
namespace tt::app::inputdialog {
uint32_t start(uint32_t callerAppInstanceId, const std::string& title,
               const std::string& message, const std::string& prefilled = "");
std::string getLastText();
}
```

5. Register it

Add a forward declaration and register the manifest at startup in `Tactility/Source/Tactility.cpp`:

```
namespace app { namespace your_app { extern const ::AppManifest manifest; } }
// ...
app_manager_add(&app::your_app::manifest);
```

Registration is what makes the app discoverable/launchable.

Inter-app communication

From `app/manager.h`:

- `app_manager_start(id, &instanceId)` — launch a plain instance.
- `app_manager_start_with_parameters(id, argc, argv, &instanceId)` — pass `argc`/`argv` to the target (app-module deep-copies them, so stack/`c_str()` temporaries are safe).
- `app_manager_start_for_result(id, parentId, argc, argv, &instanceId)` — launch a **modal child**; when it exits, the parent gets an `APP_EVENT_RESULT` carrying the child's `appMain` return value.
- Children that need to hand back more than an `int32_t` expose a "get last result" getter (e.g. `inputdialog::getLastText()`), which the parent calls after receiving the result event.

The modal-dialog pattern is the canonical example — see `InputDialog.cpp`'s `start()` (which calls `app_manager_start_for_result`) and its parent-side `getLastText()`.

Building

Add your source — no CMake edit needed for the `Tactility` layer

`Tactility/CMakeLists.txt` uses `file(GLOB_RECURSE SOURCE_FILES Source/*.c*)`, so dropping a new `.cpp` under `Tactility/Source/app/<name>/` is picked up automatically on the next configure.

Simulator (fastest iteration)

Linux/macOS only — the simulator does **not** build on native Windows:

```
cmake -B buildsim -G Ninja
ninja -C buildsim
./buildsim/Firmware/Tactility    # run the simulator
```

ESP32 firmware

```
python device.py <device-id>        # e.g. lilygo-tdeck, m5stack-cores3, cyd-2432s028r
python device.py <device-id> --dev  # optional: dev mode (4MB partition table)
idf.py build
idf.py flash monitor
```

Device IDs are the folder names under `Devices/`. On native Windows, `idf.py` isn't on PATH by default — source the ESP-IDF PowerShell profile first (see `.claude/rules/building.md` for the exact incantation).

Coding conventions

From `.claude/rules/coding-style.md`:

- **C++ layer** (`Tactility`, apps, services): `UpperCamelCase` files/types, `lowerCamelCase` functions, dirs `Source/` / `Include/` / `Private/`. Apps use namespace `tt::app::<name>`.
- No exceptions — use return types/`error_t`. Use `enum class` in C++.
- `.clang-format` is enforced (LLVM-based, 4-space indent).
- Guard ESP-only code with `#ifdef ESP_PLATFORM` in shared code (see `SystemInfo.cpp`'s heap/FAT calls).

External apps (side-loaded, brief)

For apps distributed outside the firmware (`app-framework.md` + `app/metadata.h` + `app/install.h`):

- Package a `manifest.properties` (V1 sectioned or V2 flat format) with `target_sdk`, `app_id`, `app_name`, `app_version_name`, `app_version_code`, plus an `.elf`/`.app` binary.
- Install from a tarball with `app_install(path)`, or scan a directory with `app_manager_install_path_add()` + `app_manager_install_path_scan()`.
- The SDK for building these lives under `Buildscripts/TactilitySDK/` (a CMake wrapper over the elf_loader and the module headers); it pins the ESP-IDF version and provides prebuilt `TactilityC`/`TactilityKernel`/`lvgl` libraries.